

ALGORITMOS E CÁLCULO NUMÉRICO



sagah⁺

Introdução à lógica proposicional

Alysson Nunes Diógenes

OBJETIVOS DE APRENDIZAGEM

- > Diferenciar lógica de sintaxe.
- > Identificar como é usada a tabela verdade.
- > Explicar a semântica da lógica proposicional.

Introdução

As regras da lógica matemática especificam métodos de raciocínio de afirmações matemáticas. O filósofo grego Aristóteles foi o pioneiro do raciocínio lógico, que fornece a base teórica para muitas áreas da matemática e, consequentemente, do cálculo numérico. Tem, portanto, muitas aplicações, como em projeto de computadores, em inteligência artificial, na definição de estruturas de dados para linguagens de programação, etc. (COUDERT; MADRE, 1994).

Especificamente, a lógica proposicional se preocupa com afirmações às quais os valores de verdade, “verdadeiro” e “falso”, podem ser atribuídos. O objetivo é analisar essas afirmações individualmente ou de forma composta, tarefa fundamental na matemática. Por isso, neste capítulo, apresentaremos os conceitos básicos e as principais ferramentas da lógica proposicional.

A seguir, serão examinadas as regras sintáticas que definem a linguagem da lógica proposicional. Em seguida, será introduzida a noção de atribuição de verdade, utilizada para definir o significado das sentenças da lógica proposicional. Depois, será apresentado um método mecânico para avaliar sentenças para determinada atribuição de verdade e um método para encontrar atribui-

ções de verdade que satisfaçam as frases. O capítulo é encerrado com alguns exemplos de lógica proposicional na formalização de linguagem natural e de circuitos digitais.

Conceitos básicos

A lógica proposicional se preocupa com proposições e seus inter-relacionamentos. A noção de uma proposição, aqui, não pode ser definida com precisão. Grosso modo, uma **proposição** é uma condição possível do mundo que é verdadeira ou falsa, como, por exemplo, a possibilidade de que esteja chovendo, a possibilidade de que esteja nublado e assim por diante. A condição, então, não precisa ser verdadeira para que seja uma proposição. Por exemplo, veja as frases a seguir:

1. Como você está?
2. Elefantes são menores do que formigas.
3. O Brasil é um país de dimensões continentais.
4. Vovô viu a uva.
5. Hoje não está chovendo e tenho dinheiro; logo, vou à praia.
6. $2 + 2 = 4$

Todas as frases, com exceção da primeira, são proposições. Isso pode ser verificado porque, independentemente de serem verdadeiras ou falsas, essas frases fazem uma afirmativa.

Sintaxe

Na lógica, **sintaxe** é qualquer coisa que tenha a ver com linguagens formais ou sistemas formais, sem levar em conta qualquer interpretação ou significado dado a eles. A sintaxe se preocupa com as regras usadas para construir ou transformar os símbolos e palavras de uma linguagem, em contraste com a **semântica** de uma linguagem, que se preocupa com seu significado (MENDELSON, 2009). Ou seja, a sintaxe apenas define um conjunto de símbolos, fórmulas, sistemas, teoremas, provas e interpretações, enquanto a semântica indica o sentido de frases formadas com a sintaxe.

Na ciência da computação, o termo sintaxe se refere às regras que governam a composição de expressões bem formadas em uma linguagem de programação. Como na lógica matemática, é independente da semântica e

da interpretação. Na lógica proposicional, existem dois tipos de sentenças: sentenças simples e sentenças compostas. **Frases simples** expressam fatos simples sobre o mundo, enquanto **frases compostas** expressam relações lógicas entre as frases mais simples de que são compostas.

Frases simples, na lógica proposicional, frequentemente são chamadas de **constantes proposicionais** ou, às vezes, de constantes lógicas. A seguir, escreve-se constantes de proposição como cadeias de letras, dígitos e sublinhados (“_”), em que o primeiro caractere é uma letra minúscula. Por exemplo, chover é uma constante de proposição, assim como CHOVER, c32hover e chover_ou_nevar. Chover não é uma constante de proposição porque começa com um caractere maiúsculo; 324567 não é porque começa com um número; chover-ou-nevar falha porque contém hifens (em vez de sublinhados).

As sentenças compostas são formadas a partir de sentenças mais simples e expressam relações entre as sentenças constituintes. Existem cinco tipos de frases compostas (MENDELSON, 2009):

1. negações;
2. conjunções;
3. disjunções;
4. implicações;
5. bicondicionais.

Uma **negação** consiste no operador de negação $\neg$ e em uma sentença arbitrária, chamada de alvo. Por exemplo, dada a sentença p , pode-se formar a negação de p como $(\neg p)$.

Uma **conjunção** (operador $\wedge$) é uma sequência de sentenças separadas por ocorrências do operador $\wedge$ e colocadas entre parênteses. As sentenças constituintes são chamadas de conjuntos. Por exemplo, pode-se formar a conjunção de p e q na forma $(p \wedge q)$.



Fique atento

Frases compostas podem ser formadas utilizando mais de uma conjunção. Note, por exemplo a frase composta, $p \vee (q \wedge r)$.

Uma **disjunção** (operador $\vee$) é uma sequência de sentenças separadas por ocorrências do operador $\vee$ e colocadas entre parênteses. As sentenças constituintes são chamadas de disjuntos. Por exemplo, pode-se formar a disjunção de p e q na forma: $(p \vee q)$.

Uma **implicação** (operador LOGO) consiste em um par de sentenças separadas pelo operador $\Rightarrow$ e colocadas entre parênteses. A frase à esquerda do operador é chamada de antecedente e a frase à direita é chamada de consequente. A implicação de p e q é $(p \Rightarrow q)$.

Um **bicondicional** é uma combinação de uma implicação e uma implicação reversa. Por exemplo, podemos expressar a bicondicional de p e q por $(p \Leftrightarrow q)$.

Observe que as frases constituintes em qualquer sentença composta podem ser sentenças simples ou sentenças compostas, ou, ainda, uma mistura das duas. Por exemplo, esta é uma sentença composta: $((p \vee q) \Rightarrow r)$. Uma desvantagem de nossa notação é que os parênteses tendem a se acumular e precisam ser combinados corretamente. Seria bom se pudéssemos dispensar os parênteses, por exemplo, simplificando a frase anterior para a seguinte:

$$p \vee q \Rightarrow r$$

Infelizmente, não se pode retirar inteiramente dos parênteses, uma vez que isso poderia causar confusões. Por exemplo, a frase anterior pode ter resultado da retirada dos parênteses de qualquer uma das duas seguintes frases.

$$\begin{aligned} &((p \vee q) \Rightarrow r) \\ &(p \vee (q \Rightarrow r)) \end{aligned}$$

A solução para esse problema é o uso da chamada **precedência do operador**, que é uma sequência de prioridades para operadores. A tabela a seguir fornece uma hierarquia de precedências para os operadores. O operador $\neg$ tem precedência maior do que $\wedge$; $\wedge$ tem precedência mais alta do que $\vee$; $\vee$ tem precedência mais alta do que $\Rightarrow$; e $\Rightarrow$ tem precedência maior do que $\Leftrightarrow$. Assim, a sequência de prioridades é (MENDELSON, 2009):

$\neg$	$\wedge$	$\vee$	$\Rightarrow$	$\Leftrightarrow$
--------	----------	--------	---------------	-------------------

Em frases sem parênteses, é comum que uma expressão seja ladeada por operadores, um de cada lado. Ao interpretar essas frases, a questão é se a expressão se associa ao operador a sua esquerda ou ao operador a sua direita. Pode-se usar a precedência para fazer essa determinação. Em particular, concorda-se que um operando em tal situação sempre se associa ao operador de precedência superior. Os exemplos a seguir mostram como essas regras funcionam em vários casos. As expressões à direita são as versões entre parênteses das expressões à esquerda.

$\neg p \wedge q$	$((\neg p) \wedge q)$
$p \wedge \neg q$	$(p \wedge (\neg q))$
$p \wedge q \vee r$	$((p \wedge q) \vee r)$
$p \vee q \wedge r$	$(p \vee (q \wedge r))$
$p \Rightarrow q \Leftrightarrow r$	$((p \Rightarrow q) \Leftrightarrow r)$
$p \Leftrightarrow q \Rightarrow r$	$(p \Leftrightarrow (q \Rightarrow r))$

Quando um operando está rodeado por dois operadores $\wedge$ ou por dois operadores $\vee$, o operando é associado à esquerda. Quando um operando está rodeado por dois operadores $\Rightarrow$ ou por dois operadores $\Leftrightarrow$, o operando é associado à direita, como nos exemplos a seguir.

$p \wedge q \wedge r$	$((p \wedge q) \wedge r)$
$p \vee q \vee r$	$((p \vee q) \vee r)$
$p \Rightarrow q \Rightarrow r$	$(p \Rightarrow (q \Rightarrow r))$
$p \Leftrightarrow q \Leftrightarrow r$	$(p \Leftrightarrow (q \Leftrightarrow r))$

Observe que só porque a precedência permite excluir os parênteses em alguns casos, não significa que se deve dispensá-los totalmente. Considere o exemplo mostrado anteriormente. A precedência elimina a ambiguidade ao ditar que a frase sem parênteses é uma implicação com uma disjunção como antecedente. No entanto, isso cria um problema para os casos em que se deseja expressar uma disjunção com uma implicação como disjunta. Nesses casos, deve-se manter, pelo menos, um par de parênteses.

Encerra-se a seção com duas definições simples que são úteis na discussão da lógica proposicional. Um **vocabulário proposicional** é um conjunto de constantes proposicionais. Uma **linguagem proposicional** é o conjunto de todas as sentenças proposicionais que podem ser formadas a partir de um vocabulário proposicional.



Exemplo

Construa uma frase composta utilizando as proposições:

- Deus ajuda quem cedo madruga
- Tiago acorda cedo
- Deus ajudará Tiago

Resposta:

O provérbio popular pode ser convertido em frases e em vocabulário proposicional. Para isso, pode-se reescrever as proposições da seguinte forma:

Deus ajuda quem cedo madruga **E** Tiago acorda cedo **LOGO** Deus ajudará Tiago
Assim, chamando as proposições de p , q e r na ordem em que foram listadas:

$$p \wedge q \Leftrightarrow r$$

Como usar a tabela verdade?

Antes de apresentar a tabela verdade, é importante conhecer o conceito de avaliação.

Avaliação

Avaliação é o processo de determinar os valores de verdade de sentenças compostas, dadas uma atribuição de verdade para os valores de verdade de constantes proposicionais (VAN DALEN, 2012).

De fato, existe uma técnica simples para avaliar frases complexas. Substitui-se valores verdadeiros e falsos pelas constantes de proposição na frase, formando uma expressão com 1s e 0s e operadores lógicos. Usa-se a semântica de operador para avaliar subexpressões com esses valores verdade como argumentos. Em seguida, repete-se, trabalhando de dentro para fora, até ter-se um valor verdade para a frase como um todo.

Como exemplo, considere a atribuição de verdade i mostrada a seguir.

$$p^i = 1$$

$$q^i = 0$$

$$r^i = 1$$

Usando o método de avaliação, pode-se ver que i satisfaz $(p \vee q) \wedge (\neg q \vee r)$. Note que o operador $\vee$ (OU) retorna 1 quando um ou os dois valores comparados são 1. Da mesma forma, o operador $\wedge$ (E) apenas retorna 1 quando ambos os valores comparados são 1.

$$(p \vee q) \wedge (\neg q \vee r)$$

$$(1 \vee 0) \wedge (\neg 0 \vee 1)$$

$$1 \wedge (\neg 0 \vee 1)$$

$$1 \wedge (1 \vee 1)$$

$$1 \wedge 1$$

$$1$$

Agora considere a atribuição de verdade j definida como segue:

$$p^j = 0$$

$$q^j = 1$$

$$r^j = 0$$

Nesse caso, j não satisfaz $(p \vee q) \wedge (\neg q \vee r)$:

$$(p \vee q) \wedge (\neg q \vee r)$$

$$(0 \vee 1) \wedge (\neg 1 \vee 0)$$

$$1 \wedge (\neg 1 \vee 0)$$

$$1 \wedge (0 \vee 0)$$

$$1 \wedge 0$$

$$0$$

Usando essa técnica, pode-se avaliar a verdade de sentenças arbitrárias em uma linguagem. O custo é proporcional ao tamanho da frase. Claro que, em alguns casos, é possível economizar e fazer ainda melhor. Por exemplo, ao avaliar uma conjunção, se for descoberto que o primeiro conjunto é falso, então não há necessidade de avaliar o segundo conjunto, pois a sentença como um todo deve ser falsa (VÄÄNÄNEN, 2007).

Satisfação

A **satisfação** é o oposto da avaliação (VAN DALEN, 2012). Começa-se com uma ou mais sentenças compostas e tenta-se descobrir quais atribuições de verdade satisfazem essas sentenças. Uma boa característica da lógica proposicional é que existem procedimentos eficazes para encontrar atribuições de verdade que satisfazem sentenças da lógica proposicional. Nesta seção, estuda-se um método baseado em tabelas verdade.

Uma **tabela verdade** para a linguagem proposicional é uma tabela que mostra todas as atribuições de verdade possíveis para as constantes proposicionais na linguagem. As colunas da tabela correspondem às constantes de proposição da linguagem e as linhas correspondem a diferentes atribuições de verdade para essas constantes.

Veja, a seguir, uma tabela verdade para uma linguagem proposicional com apenas três constantes de proposição (p , q e r). Cada coluna corresponde a uma constante de proposição e cada linha corresponde a uma única atribuição de verdade.

p	q	r
1	1	1
1	1	0
1	0	1
1	0	0
0	1	1
0	1	0
0	0	1
0	0	0

Observe que, para uma linguagem proposicional com n constantes de proposição, há n colunas na tabela verdade e 2^n linhas.

Na solução de problemas de satisfação, começa-se com uma tabela verdade para as constantes proposicionais da linguagem. Em seguida, processa-se as sentenças sucessivamente, para cada sentença, colocando um “x” próximo às linhas na tabela verdade correspondente a atribuições de verdade que não satisfazem a sentença. As atribuições de verdade restantes no final desse processo são todas as atribuições de verdade possíveis das sentenças de entrada.

Como exemplo, considere a sentença $p \vee q \Rightarrow q \wedge r$. Pode-se encontrar todas as atribuições de verdade que satisfazem essa sentença construindo uma tabela de verdade para p, q e r . Veja na tabela a seguir. Colocou-se um “x” próximo a cada linha que não satisfaça a sentença (linhas 2, 3, 4, 6). Finalmente, considera-se as linhas restantes (1, 5, 7, 8) como respostas.

	p	q	r	
	1	1	1	
x	1	1	0	x
x	1	0	1	x
x	1	0	0	x
	0	1	1	

(Continua)

(Continuação)

	p	q	r	
x	0	1	0	x
	0	0	1	
	0	0	0	

A desvantagem do método da tabela verdade é a complexidade computacional. Conforme mencionado, o tamanho de uma tabela verdade para uma linguagem cresce exponencialmente com o número de constantes proposicionais na linguagem. Quando o número de constantes é pequeno, o método funciona bem. Quando o número é grande, o método tende a se tornar impraticável. Mesmo para problemas de tamanho moderado, pode ser entediante. Mesmo para uma aplicação simples em que existam 16 constantes de proposição, existem 65.536 atribuições de verdade (BICEN, 2020).

Ao longo dos anos, os pesquisadores propuseram maneiras de melhorar o desempenho da verificação da tabela verdade. No entanto, a melhor abordagem para lidar com grandes vocabulários é usar manipulação simbólica (ou seja, raciocínio lógico e testes) em vez de verificação da tabela verdade (VAN DALEN, 2012).



Saiba mais

Tabelas verdade também podem ser utilizadas para otimização de circuitos de portas lógicas.

Semântica

Diferentemente da sintaxe, que não se preocupa com o sentido de uma frase, a semântica é o ramo da lógica que trata do sentido que das frases. O tratamento da semântica em lógica é semelhante a seu tratamento em álgebra. A álgebra não se preocupa com a importância das variáveis no mundo real. O que é interessante são as relações entre os valores das variáveis expressas nas equações que escrevemos. Os métodos algébricos são projetados para respeitar essas relações, independentemente do que as variáveis representam (VAN DALEN, 2012).



Fique atento

A semântica se preocupa com o uso correto dos operadores para que uma frase faça sentido.

De maneira semelhante, a lógica não se preocupa com o significado das constantes proposicionais no mundo real. É curioso notar a relação entre os valores de verdade de sentenças simples e os valores de verdade de sentenças compostas dentro das quais as sentenças simples estão contidas. Assim como acontece com a álgebra, os métodos de raciocínio lógico são independentes do significado das constantes proposicionais — tudo o que importa é a forma das frases.

Embora os valores atribuídos às constantes de proposição não sejam cruciais no sentido que acabamos de descrever, ao falar da lógica, às vezes é útil tornar explícitas as atribuições de verdade e considerar várias atribuições ou todas as atribuições e assim por diante. Essa atribuição é chamada de **atribuição de verdade**.

Formalmente, uma atribuição de verdade para um vocabulário proposicional é uma função que atribui um valor de verdade a cada uma das constantes proposicionais do vocabulário. A seguir, usamos o dígito 1 como sinônimo para verdadeiro e 0 como sinônimo para falso, e nos referimos ao valor de uma constante ou expressão sob uma atribuição de verdade i sobrescrito da constante ou expressão com i como sobrescrito.

A atribuição mostrada a seguir é um exemplo para o caso de um vocabulário proposicional com apenas três constantes de proposição: p , q e r .

$$p^i = 1$$

$$q^i = 0$$

$$r^i = 1$$

A seguinte atribuição é outra atribuição verdadeira para o mesmo vocabulário:

$$p^i = 0$$

$$q^i = 0$$

$$r^i = 1$$

Observe que as fórmulas anteriores não são sentenças em si mesmas na lógica proposicional. a lógica proposicional não permite sobrescritos e não usa o símbolo $=$. Em vez disso, essas são declarações informais sobre atribuições de verdade específicas. Embora falar sobre lógica proposicional usando uma notação semelhante a essa lógica proposicional possa, às vezes, ser confuso, isso permite transmitir metainformação com precisão e eficiência. Para minimizar problemas, usa-se essa metanotação com pouca frequência e apenas quando há pouca chance de confusão.

Olhando para as atribuições de verdade anteriores, é importante ter em mente que, no que diz respeito à lógica, qualquer atribuição de verdade é tão boa quanto qualquer outra. A própria lógica não fixa a atribuição de verdade de constantes de proposições individuais.

Por outro lado, dada uma atribuição de verdade para as constantes proposicionais de uma linguagem, a lógica fixa a atribuição de verdade para todas as sentenças compostas nessa linguagem. Na verdade, é possível determinar o valor verdade de uma frase composta aplicando repetidamente as seguintes regras.

Se o valor verdade de uma frase é “verdadeiro”, o valor de verdade de sua negação é “falso”. Se o valor de verdade de uma frase é “falso”, o valor de verdade de sua negação é “verdadeiro”:

ϕ	$\neg\phi$
1	0
0	1

O valor verdade de uma conjunção é “verdadeiro” se, e somente se, os valores de verdade de seus conjuntos forem ambos “verdadeiros”; caso contrário, o valor verdade é “falso”:

ϕ	ψ	$\phi \wedge \psi$
1	1	1
1	0	0
0	1	0
0	0	0

O valor verdade de uma disjunção é “verdadeiro” se, e somente se, o valor de verdade de pelo menos uma das disjunções for “verdadeiro”; caso contrário, o valor verdade é “falso”. Observe que essa é a inclusiva ou interpretação do operador $\vee$ e é diferenciada da exclusiva ou interpretação na qual uma disjunção é verdadeira se, e somente se, um número ímpar de seus disjuntos for “verdadeiro”:

Φ	Ψ	$\Phi \vee \Psi$
1	1	1
1	0	1
0	1	1
0	0	0

O valor verdade de uma implicação é “falso” se, e somente se seu antecedente for “verdadeiro” e seu conseqüente for “falso”; caso contrário, o valor verdade é “verdadeiro”. Isso é chamado de **implicação material** (VÄÄNÄNEN, 2007):

Φ	Ψ	$\Phi \Rightarrow \Psi$
1	1	1
1	0	0
0	1	1
0	0	1

Uma bicondicional é “verdadeira” se, e somente se, os valores “verdadeiros” de seus constituintes concordarem, ou seja, se forem “verdadeiros” ou “falsos”:

Φ	Ψ	$\Phi \Leftrightarrow \Psi$
1	1	1
1	0	0
0	1	0
0	0	1

Referências

BICEN, Y. Propositional logic concept for fault diagnosis in complex systems. *Engineering Science and Technology, an International Journal*, v. 23, nº 5, p. 1068–1073, out. 2020. Disponível em: <https://www.sciencedirect.com/science/article/pii/S2215098619318324>. Acesso em: 25 nov. 2020.

COUDERT, O.; MADRE, J. C. Metaprime: an interactive fault-tree analyzer. *IEEE Transactions on Reliability*, v. 43, nº 1, p. 121–127, mar. 1994.

MENDELSON, E. *Introduction to mathematical logic*. 5th ed. London: Chapman and Hall, 2009.

VÄÄNÄNEN, J. *Dependence logic: a new approach to independence friendly logic*. Cambridge: Cambridge University Press, 2007.

VAN DALEN, D. *Logic and structure*. Berlin: Springer, 2012.

Leitura recomendada

ALENCAR FILHO, E. de. *Iniciação à lógica matemática*. 21. ed. Barueri: Nobel, 2017.



Fique atento

Os links para sites da web fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais links.

Conteúdo:

sagah⁺